Ders 24 Genişletilmiş Notu: Dizgi Arama: Kaba Kuvvet, Boyer-Moore ve Trie

Ders 24 Genişletilmiş Notu: Dizgi Arama: Kaba Kuvvet, Boyer-Moore ve Trie

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dizgi Arama Problemi
- 2. Kaba Kuvvet Aramanın Mantığı
- 3. Kaba Kuvvette En İyi ve En Kötü Durum
- 4. Kaba Kuvvet Algoritmasını Tasarlamak
- 5. Ödev Sorusu Üzerine: PGM Biçimi ve Run-Length Kodlu Veri
- 6. Boyer-Moore Yaklaşımının Fikri
- 7. Ne Kadar Atlayacağız?
- 8. En Sağdaki Geçiş Tablosu
- 9. Boyer-Moore Kodu
- 10. Örnek Üzerinde Adım Adım İzleme
- 11. Boyer-Moore Karmaşıklığı
- 12. Trie ve Otomatik Tamamlama

1. Dizgi Arama Problemi

Hoca dönemin bu son dersini, kendi sözüyle çok zorlayıcı olmayan bir konuya ayırdı: alt dizgi arama (substring search). Konuya girerken dersin en başındaki hedefi hatırlattı. Bu derste en çok ihtiyaç duyulan temel algoritmalar, yani sıralama ve arama algoritmaları ile bunların tasarımında kullanılan veri yapıları anlatılacaktı. Listeler, ağaçlar, graflar, yığınlar ve kuyruklar görüldü. Sıra, arama çeşitlerinin en çok kullanılanlarından birine geldi: bir metin (text) içinde bir dizgi (string) aramak.

İkili arama (binary search) gibi yöntemleri incelerken hep tek bir bilgi, yani bir sayı aranıyordu. Bir sayı dizisinde 378'i arıyorsak o tek sayıyı bulduğumuz an işimiz biter. Dizgi aramada ise bir diziyi başka bir bilginin içinde ararız. Aranan dizinin **tamamının** metinde karşılığı olmalıdır. Hedef yine olabildiğince etkili bir çözüm bulmaktır.

Bu problem için örneklerde sık görülen hoş bir ifade vardır: samanlıkta iğne aramak. Hocanın ilk örneğinde metin aşağıdadır, aranan desen (pattern) ise `NEEDLE`'dır:

```
metin : FINDINAHAYSTACKNEEDLEINA
desen : NEEDLE   (6 karakter)
```

Hoca, örneklerde büyük harf kullansa da karakterlerin harf olmak zorunda olmadığını belirtti. Ünlem, soru işareti ya da rakam da olabilir. Metnin bir yazı olması bile gerekmez. Önemli olan, karakter türünde bir bilgi içinde bir dizgi aramamızdır.

> **Ders sonrası not:** Derste metnin uzunluğu için 22 karakter denildi. Yukarıdaki metin 24 karakterdir (python ile sayıldı, indisler 0-23).

2. Kaba Kuvvet Aramanın Mantığı

En temel çözüm kaba kuvvet (brute force) yaklaşımıdır. Hoca kaba kuvveti, problemin en kaba ve en temel hâliyle çözülmesi olarak tanımladı. Bir öğrencinin de söylediği gibi iç içe döngülerle baştan sona kontrol edilir.

Desen metnin başına yerleştirilir. `N`, `F` ile karşılaştırılır, olmaz. Desen bir sağa kaydırılır, `N` bu kez ikinci karakterle karşılaştırılır. Bu böyle sürer. Bir yerde `N` tutarsa ikinci harfe, yani `E`'ye bakılır; o da tutarsa üçüncü harfe geçilir. Desenin bütün harfleri tek tek eşleşirse aradığımız bulunmuştur. Bu örnekte şans eseri uyuşmazlık neredeyse hep ilk karakterde çıkıyordu.

Peki birkaç karakter tutuyorsa ne olur? Hoca bunu görmek için `ABRA` desenini içeren ikinci bir örnek verdi. İlk hizalamada `A` ile `A`, `B` ile `B` uyuşur, ardından üçüncü karakter bozulur. Burada durulur.

Sorulması gereken soru şudur: işe nereden devam etmeliyiz? Cevap, başladığımız konumun bir sağıdır, yani bir numaralı adres. Çünkü arada bir yerde `ABRA` başlıyor olabilir ve daha ileri atlarsak onu kaçırırız. Hoca tahtada bir hizalamayı atlayınca bir öğrenci hemen uyardı ve hoca adımı düzeltti; tek tek ilerlemenin ne kadar kritik olduğu böylece de görüldü.

> **Hocanın vurgusu:** Karakterler uyuştuğu sürece bakmaya devam ederiz. Bozulduğu yerde, işe başladığımız noktanın bir ilerisinden devam ederiz. `i`'den başladıysak ve herhangi bir adımda tutmadıysa `i+1`'den devam etmek zorundayız. Bozulduğu yerden devam etmek gibi bir lüksümüz yoktur.

3. Kaba Kuvvette En İyi ve En Kötü Durum

3.1 En İyi Durum

Metnin uzunluğu n, desenin uzunluğu m olsun. İlk örnekte olduğu gibi şansımıza her seferinde birinci ya da ikinci karakterde bozuluyorsa toplamda yaklaşık n karşılaştırma yaparız. Hoca bunun fena olmadığını söyledi. Ama `ABRA` örneğinde gördüğümüz gibi üç karakter gidip başa dönmek şüphe uyandırır. Metin ve desen tekrar eden harflerden oluşuyorsa durum daha da kötüleşir; hocanın örneği DNA üzerinde dizgi aramaktır.

> **Ders sonrası not:** Hocanın "en iyi durum n" sözü, desenin bulunmadığı ya da geç bulunduğu aramada uyuşmazlığın hep ilk karakterde çıktığı durumu anlatır. Bu durumda karşılaştırma sayısı desen bulunmazsa tam $n - m + 1$'dir. Desen metnin en başındaysa arama m karşılaştırmayla biter.

3.2 En Kötü Durum

Hoca en kötü durum için şu tekrarlı örneği yazdı:

```
metin : A A A A A A A A A B
desen : A A A A B
        her hizalamada dört A uyuşur, beşinci karşılaştırma B'de bozulur;
        desen bir kaydırılır ve aynı şey tekrarlanır
```

"İnsan Google'da böyle bir şeyi ne zaman arar?" denebilir. Hocanın cevabı, burada amacın en kötü durumu ortaya koymak olduğudur, çünkü çözümleri hep en kötü duruma göre düşünürüz. Günlük hayatta metin ararken kaba kuvveti kullanabilirsiniz. Ama en kötü durumun olma ihtimalinin yüksek olduğu kritik işlerde daha iyi bir çözüm üretmeniz gerekir.

Bu örnekte her hizalamada desenin tamamı karşılaştırılır. Hizalama sayısı yaklaşık n, her hizalamadaki karşılaştırma m olduğundan en kötü durum $O(n \cdot m)$ olur. Hoca, fazlalıkları atmanın bakış açımız olduğunu hatırlatarak sonucu $n \cdot m$ olarak bıraktı.

Örnekteki sayılarla izlersek (ara adım eklendi): $n = 10$, $m = 5$ için başlangıç konumları $i = 0, 1, \ldots, 5$ olur, yani 6 hizalama vardır. İlk beşinde dört `A` uyuşup beşinci karakter bozulur. Sonuncusunda ($i = 5$) desen bulunur. Her hizalama 5 karşılaştırma tutar, toplam $6 \cdot 5 = 30$ karşılaştırma eder.

> **Ders sonrası not:** Hoca en kötü durum için "$(n - m - 1)$ çarpı n" dedi; doğrusu $(n-m+1) \cdot m$'dir. Son başlangıç konumu $n-m$ olduğundan 0 ile birlikte $n-m+1$

hizalama vardır ve her birinde en fazla m karşılaştırma yapılır. Yukarıdaki örneğin 30 sonucu, sayaçlı bir çalıştırmayla bu formülle aynı çıktı.

3.3 m Sabit Sayılabilir mi?

Hoca burada özellikle bir şey hatırlatmak istedi. $n \cdot m$ dediğimizde m'yi nasıl değerlendireceğiz? Karmaşıklık $3n$ olsaydı, yani $m = 3$ olsaydı buna rahatça n derdik. $23n$ görsek ona da n derdik. Çünkü n sonsuza giderken sabit katsayıları hesaba katmanın anlamı yoktur.

Ama m büyüdükçe, örneğin 1 milyon olduğunda, "1 milyon çarpı n" artık küçük katsayılı bir ifade değildir ve yavaş yavaş n^2'ye doğru gider. Burada sorun, m'nin belirsiz olması, yani önceden kestirilememesidir. Çok büyük bir belgede, örneğin bir kitaba ait dosyada büyük bir metin parçası arıyor olabilirsiniz.

> **Hocanın vurgusu:** Çalıştığınız problemdeki durumları biliyor ve m'nin en fazla nereye kadar çıkabileceğinden eminseniz $O(n)$ demeniz doğrudur. Değilseniz karmaşıklığı $O(n \cdot m)$ olarak belirtmek zorundasınız.

4. Kaba Kuvvet Algoritmasını Tasarlamak

Kaba kuvvet çok rahat tasarlanabilecek bir algoritmadır. Hoca bunu adım adım kurdu ve yol boyunca genel tasarım tavsiyeleri verdi. Gerçek bir programda metin ve deseni dizgi olarak alıp uzunluklarını `strlen` ile bulacağımızı, burada ise algoritmik olarak anlattığını söyledi.

4.1 Dış Döngü Nereye Kadar Gider?

Hoca örnek olarak `KARGAGARDEDI` metnini ve `GAK` desenini kullandı. Burada $n = 12$, $m = 3$ olur. Soru şudur: `i` nereye kadar gitmelidir? Bir öğrenci doğru cevabı verdi: $n - m$'ye kadar.

En son baktığımız yer, `GAK` desenini `E` karakterine oturttuğumuz konumdur: $12 - 3 = 9$. `E` ile `G` karşılaştırılır, tutmazsa iş biter. Daha ileri gitmenin anlamı yoktur, çünkü sonda üç karakter kalmadığına göre bulma şansımız yoktur.

4.2 Döngü Sınırını Döngü Dışında Hesaplamak

Hoca döngü sınırı için ayrı bir değişken tanımladı: `NN = N - M`. Bunun nedenini ikinci sınıf öğrencilerine özellikle anlatmak istedi. Döngü `for (i = 0; i <= N - M; i++)` diye yazılsaydı, birçok derleyici bunu kendisi optimize ederdi. Derleyici `N` ve `M`'nin döngü içinde değişmediğini görür, `N - M`'nin değerini (örnekte 9) bir kez hesaplar ve hep onu kullanır.

Ama bunu her derleyici yapmaz. Optimize etmezse her adımda belleğe gider, `N`'e erişir, `M`'e erişir ve çıkarmayı yeniden yapar. Örneğin 20'den 5'i çıkarıp 15 bulur, "daha 15 olmamış, devam edeyim" der. Derleyicinin ne yaptığını bilmiyorsanız en iyisi tek satırlık bir komutla değeri bir değişkene atayıp onu kullanmaktır. Çarpma gibi daha pahalı işlemlerde durum daha da kötüdür: `i <= N * M` yazarsanız her turda çarpma yapılabilir.

> **Hocanın vurgusu:** Artık yalnız algoritmik optimizasyona değil, gereksiz işlem yapmamaya da dikkat etmelisiniz. Tasarımınız bittikten sonra bir tur daha üzerinden geçip gözünüze çarpan bu tür işlemleri optimize edin.
>
> **Ek bilgi (derste anlatılmadı):** Döngüde değeri değişmeyen bir ifadeye döngüden bağımsız (loop-invariant) ifade, onu döngü dışına çıkarmaya da döngüden bağımsız kod taşıma (loop-invariant code motion) denir. Hocanın bahsettiği derleyici optimizasyonu budur.

Soru: For döngüsü içinde n kez çıkarma yapmak mı daha masraflı, yoksa `NN` değerini fonksiyon bitene kadar bellekte tutmak mı?

Cevap: n kez `N - M` işlemi yapmak daha masraflıdır. Tek bir değişkeni bir kez hesaplar, sonra yalnız ona erişirsiniz. Bu değişken 4 bayt kadar yer kaplar, megabaytlarca değil; "4 bayttan kimse ölmez". Oysa döngü 1 milyon ya da 10 milyon kez dönüyorsa her turda çıkarma, hele çarpma yapmak daha pahalıdır. Çoğu derleyici bu işi zaten kendisi yapar. Öneri, optimizasyon

yapmayan bir derleyicide çalışıyorsanız ya da yapıp yapmadığından emin değilseniz ve hız sizin için çok önemliyse (örneğin gömülü sistemde) geçerlidir.

Sohbette gelen "gömülü sistemlerde tutmamak daha avantajlı olur" görüşüne hoca ara vermeden önce ayrıca cevap verdi: 4 baytlık bir değişken gömülü sistemde de dert olmaz. Orada hız daha büyük sorundur, bu yüzden değeri bir kez hesaplayıp tutmak daha avantajlıdır.

4.3 İç Döngü

Hocanın tasarım tavsiyesi, algoritmayı önce elle bir örnek üzerinde denemektir. Herkesin kendi tarzı olsa da ona göre en rahat yol budur. Metindeki indis `i`, desendeki indis `j` olsun. `i`, işin başladığı metin konumunu tutar. `j` ise desende 0. karakterden başlar.

`G`'yi yerine koyduk: `G` ile `G` aynı, `j` bir artar. `A` ile `A` aynı, `j` bir artar. Şimdi `K` ile `R` karşılaştırılır ve bozulur. Bu kez `G`'yi bir sonraki konumdaki `A`'nın karşısına oturtmamız gerekir. Karakterler aynı olsaydı, yani metinde gerçekten `GAK` geçseydi, `j` desenin sonuna gelirdi. İç döngünün iki şartı böylece ortaya çıkar:

- `j < M`: desen bitene kadar dön. `j == M` olması, desenin bütün karakterlerine bakılıp bulunduğu anlamına gelir.
- `pat[j] == text[i+j]`: eşitlik bozulana kadar dön. Bozulduktan sonra devam etmenin anlamı yoktur.

  Hocanın vurgusu: Burada `i`'yi ilerletmiyoruz. `i`'yi `i0`, `i1`, `i2` diye ilerletip tutmayınca geri almak anlamsız olur. Yalnız `j` ilerler ve metnin `i+j`'inci karakteriyle karşılaştırma yapılır.

4.4 Döngüden Neden Çıktık?

İç döngüden çıkınca ilk soru şudur: niye çıktık? Bir öğrencinin dediği gibi `j`'yi kontrol ederiz. `j == M` ise desen bulunmuştur, değilse eşitlik bozulmuştur.

Hoca işe fonksiyon yazar gibi başlamıştı. Fonksiyon olsaydı `j == M` olunca `return` deyip çıkabilirdi. Ama akış diyagramında bu hoş görünmeyeceği için döngüden "yasal yollarla" çıkmak üzere bir `found` bayrağı kullandı. Bulununca `found = 1` yapılır, bulunamayınca `i` artırılır.

Tahtadaki akış diyagramının adımları şöyledir:

```
oku: N, M, text[N], pat[M]
NN = N - M
found = 0, i = 0
while (!found && i <= NN)
    j = 0
    while (j < M && pat[j] == text[i+j])
        j++
    if (j == M)  found = 1        // desen bulundu
    else         i++              // bulunamadı, bir sağa kay
if (found)  yaz i
else        yaz "Bulunamadı"
```

Aşağıda aynı akışın çalışan C hâli vardır. Hoca akış diyagramı çizdi. `main` ve `strlen` ile uzunluk alma, derlenebilmesi için eklendi:

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char text[] = "KARGAGARDEDI", pat[] = "GAK";
    int N = strlen(text), M = strlen(pat);
    int NN = N - M;
    int found = 0, i = 0, j;

    while (!found && i <= NN)
```

```
    {
        j = 0;
        while (j < M && pat[j] == text[i + j])
            j++;
        if (j == M)
            found = 1;
        else
            i++;
    }
    if (found)
        printf("%d\n", i);
    else
        printf("Bulunamadi\n");
    return 0;
}
```

Bu program `gcc -Wall` ile uyarısız derlendi ve `Bulunamadi` yazdı. Metinde `GAK` geçmediği için bu doğru sonuçtur.

İki kısa izleme hocanın anlatımından alınmıştır:

- `KARGAGARDEDI` ve `GAK`, $i = 5$ (ikinci `G`): `G`-`G` ve `A`-`A` uyuşur, `R`-`K` bozulur. Arama `i = 6`'dan (`A`) devam eder, yani `G` bu kez `A`'nın karşısına gelir.
- Metin `KAR...`, desen `KAL`, $i = 0$: $j = 0$ için `K`-`K`, $j = 1$ için `A`-`A` uyuşur, $j = 2$ için `R`-`L` uyuşmaz. Döngüden çıkılır. $j = 2 = m - 1$ olduğundan $j \neq m$'dir. `i` bir artar, $i = 1$ olur ve sıradaki adımda `K`, `A` ile karşılaştırılır.

Kaba kuvvetin sorunu buradadır: tekrarın çok olduğu metinlerde ya da uyuşmazlığın sonlara doğru çıktığı durumlarda, hatta genel olarak yarı yola gidip bozulan aramalarda verimli değildir.

5. Ödev Sorusu Üzerine: PGM Biçimi ve Run-Length Kodlu Veri

Aradan dönüşte Boyer-Moore'a geçmeden önce öğrencilerin ödev sorularıyla ilgilenildi. Konu dizgi arama dışındadır. Fakat dosya biçimleri ve sıkıştırılmış veri üzerinde çalışma açısından akademik içerik taşıdığı için burada aktarılıyor.

5.1 Metin ve İkili PGM

PGM görüntü biçiminin iki türü vardır: metin (text, ASCII) ve ikili (binary). Hoca farkı programlama dillerinden hatırlattı. Metin dosyasında boşluk, satır sonu gibi özel karakterler bulunur. İkili dosyada ise bunlar yoktur; sayılar dip dibe durur ve `fread` ile blok olarak okunur.

Hoca genelde ikili PGM'lerle çalıştığını, metin biçimini uzun süredir görmediğini söyledi. Okuma tarafında ikisini ayırmak zor değildir: biçim koduna bakan bir `if`, P2 ise şunu, P5 ise bunu yapar. Yazarken ise ikili biçimi önerdi, çünkü blok olarak yazmak daha kolaydır. Bir öğrenci ödev bağlantısındaki belgede P2'nin metin, P5'in ikili olduğunu okudu; ayrıca P2 başlığında satır ve sütun sayısından sonra bir de en büyük değer bulunduğunu gösterdi.

> **Ders sonrası not:** Hoca belgeye bakarken bir an "P4 ikiliymiş" dedi; öğrencinin de okuduğu gibi doğrusu P2 metin, P5 ikili PGM'dir. P4 büyük olasılıkla PGM değil, siyah-beyaz PBM biçiminin ikili kodudur. Ayrıca P2 başlığında önce satır sayısı, sonra sütun sayısı geldiği söylendi; Netpbm biçimlerinde büyük olasılıkla önce genişlik (sütun), sonra yükseklik (satır) yazılır.

5.2 Run-Length Kodlu Görüntüde Piksel Değiştirme

Bir öğrenci, sıkıştırılmış dosya üzerindeki işlemlerin "açmadan" yapılması isteğini sordu. Hoca kastın dosyayı açmamak değil, kodlamayı çözmemek (decode etmemek) olduğunu açıkladı. Dosyayı açmadan zaten hiçbir şey yapılamaz.

Ödevdeki örnek görüntü 5 satır, 3 sütundur. Run-length verisi şöyledir: 10 tane 8, 3 tane 20, 2 tane 6. Toplam $10 + 3 + 2 = 15 = 5 \cdot 3$ piksel eder. İstenen, 3. satır 1. sütundaki pikseli 90 yapmaktır. Satır ve sütunlar 0'dan sayılır.

Hocanın kastettiği yöntem adım adımdır:

- **Pikselin sırası:** Satır başına 3 sütun olduğu için piksel sırası $3 \cdot 3 + 1 = 10$ olur (ara adım eklendi). Hoca bunu tahtada sayarak buldu. 0'dan sayınca 10., 1'den sayınca 11. pikseldir.
- **Hangi run:** Run uzunlukları baştan toplanır. İlk run 0-9 arasını kaplar, demek ki bu run atlanır. İkinci run 10-12 arasını kaplar. Aranan piksel 3 tane 20'nin ilkidir.
- **Bölme:** O run bölünür: `3 20` yerine `1 90 2 20` yazılır.

```
önce : 10 8   3 20          2 6
sonra: 10 8   1 90   2 20   2 6      (10 + 1 + 2 + 2 = 15 piksel)
```

Görüntüyü matrise açmaya gerek yoktur. Hoca, bu matrisin 1000×1000 olabileceğini hatırlattı: Yalnız bir yerde iş yapacakken her şeyi açıp sonra mecburen hepsini yeniden sıkıştırmak gereksizdir.

Bir öğrenci açmadan yapmanın iki zorluğunu söyledi. Birincisi, dosyaya yazınca üzerine yazılmasıdır. Hocanın cevabı, zaten dosya üzerinde hiç çalışılmamasıdır:

> **Hocanın vurgusu:** Hiçbir zaman dosyanın üzerinde iş yapmayın, dosya giriş/çıkışı yavaştır. Satır sayısı, sütun sayısı ve veriyi dosyadan belleğe ayrılan bir diziye alın. Matris açmanıza gerek yok, işlemleri bu dizi üzerinde yapın. İş bitip kullanıcı çıkışı seçince sonucu dosyaya geri yazın.

İkinci zorluk komşu run'larla ilgilidir. Değiştirilen pikselin komşusunda aynı değer varsa, örneğin `10 90` varken yanına `1 90` eklenirse, doğru kodlama `11 90` olmalıdır. Açıp yeniden sıkıştırınca bu kendiliğinden düzelir. Hoca da doğru sonucun `11 90` olması gerektiğini kabul etti. Bunun nasıl yapılacağına karışmadı: Veri geçici bir yere alınıp yanına eklenebilir ya da araya eleman yerleştirilebilir.

6. Boyer-Moore Yaklaşımının Fikri

Dizgi arama için birçok algoritma vardır. Hoca bunlardan çok kullanılan ve güzel bir tanesini, adlandırmasıyla Boyer-Moore-Horspool algoritmasını anlatacağını söyledi. Boyer ve Moore iki kişidir ve algoritmayı ortaya koyanlardır. Hocaya göre Horspool 1980 civarında algoritmayı biraz daha verimli hâle getirmiştir, mantık aynı kalır. Hoca, birilerinin aklına böyle değişik fikirler gelmesinin çok eğlenceli olduğunu da ekledi.

6.1 Sondan Başa Karşılaştırmak

Temel fark şudur: karşılaştırma desenin **sonundan başına** doğru, yani sağdan sola yapılır. Kaba kuvvette yön soldan sağaydı.

Boyer ve Moore'un sorusu şudur: kaba kuvvette neden tek tek ilerliyorduk? Arada bir yerde `NEEDLE` başlıyorsa onu kaçırmamak için. Bu korkudan kurtulmak için şöyle düşünürler: `E` ile `N` uyuşmadıysa, desenin içinde `N` var mı diye bakarım. Varsa `N`'ler çakışacak şekilde deseni yerleştiririm. Böylece artık kaçırmam. Hoca bunu "çok akıllıca" diye nitelendirdi.

6.2 İlk Örnek: Sezgisel Geçiş

Aynı metin ve `NEEDLE` deseniyle hocanın gösterdiği adımlar şöyledir:

- `E` ile `N` karşılaştırılır, aynı değil. Desende `N` var. `N`'ler çakışacak şekilde desen kaydırılır. Şimdi `E`, `S`'nin karşısına gelir.
- `E` ile `S` aynı değil. Desende `S` yok. `N`'leri çakıştırarak oturttuğumuza göre, burada `NEEDLE` geçiyor olsaydı bu konumda `E` olması gerekirdi. Olmadığına göre bu bölgede desen olamaz. Hop diye atlanır.
- `E` ile `E` uyuşur. Sonra `L` ile `N` karşılaştırılır, bozulur. Desende `N` var, `N`'ler çakıştırılır.
- `E`, `L`, `D`, `E`, `E`, `N` sırayla sondan başa karşılaştırılır, hepsi uyuşur. Desen bulunmuştur.

Hoca “oturtmak” sözünün ne anlama geldiğini de açıkladı: Bir şeyi bir yere gerçekten koymuyoruz. Yalnız, `i`’yi ne kadar kaydıracağımıza ve son karakteri metinde neyle karşılaştıracağımıza karar veriyoruz.

Soru: Bir karakter desende iki kez geçseydi sağdakine mi oturtacaktı?

Cevap: Bu önemli bir nokta. `NEEDLE`’da üç `E` var, hangisi seçilecek? Tek tek incelenecek (Bölüm 7.2).

6.3 Temel Döngü

`i` 0’dan başlar, `j` ise `M-1`’den başlar, çünkü geriye doğru gideceğiz:

```
j = M - 1
while (j >= 0 && p[j] == t[i+j])
    j--
```

Kaba kuvvetle aynı mantık tersten kurulmuştur. Şu anda açıkta kalan tek soru, eşitlik bozulduğunda ne yapılacağıdır.

Şansımıza her seferinde m karakter atlayabildiğimizi varsayalım. Hoca buna “en iyi durum demeyeyim” diye ekleyerek, zıplaya zıplaya gidebildiğimizde en sonda bulsak bile karmaşıklığın n/m olduğunu söyledi. Tabii “hayat bu kadar kolay değil”, kolay olsaydı kaba kuvveti kullanırdık zaten.

7. Ne Kadar Atlayacağız?

Uyuşmazlık olduğunda soru şudur: ne kadar sıçrayacağız (how much to skip)? Hoca iki temel durum ayırdı: çakışmayı bozan karakter desende yok ve desende var. Metindeki bozan karakter `c = t[i+j]` olsun.

7.1 Birinci Durum: Karakter Desende Yok

Bu en kolay durumdur. Hocanın kurduğu örnekte başlangıçta $i = 8$, $j = 5$’tir ve `E` ile `S` uyuşmaz. `S` desende yoktur. Yeni başlangıç, `S`’nin hemen sağıdır:

$$\text{skip} = j + 1 = 5 + 1 = 6, \qquad i = 8 + 6 = 14$$

İkinci adımda $i = 14$’tür. `E` ile `E` uyuşur, $j = 4$’te `Y` ile `L` uyuşmaz. `Y` de `NEEDLE` içinde yoktur:

$$\text{skip} = j + 1 = 4 + 1 = 5, \qquad i = 14 + 5 = 19$$

İlk adımda uyuşmazlık desenin son karakterinde olduğu için $j+1 = m$ çıkar ve sıçrama desen boyu kadardır. İkinci adım, sıçramanın her zaman m olmadığını gösterir. Hedef hep aynıdır: arada başlayan bir eşleşmeyi kaçırmamak. Karakter desende geçmiyorsa, onu kapsayan hiçbir hizalamada desen eşleşemez. Ama hemen sonrasından başlayan bir `NEEDLE` olabilir. Uyuşmayan karakter `i+j`’inci konumdaysa işe `i+j+1`’den devam etmeliyiz, bu da skip $= j+1$ demektir.

Soru: Neden `Y` ile baktık? `E` ile bakmamız gerekmiyor mu?

Cevap: Önce `S` ile `E` çakışmadı ve `i` 14 oldu. Bu hizalamada `E` ile `E` aynıdır, ondan sonra `L` ile `Y` çakışmaz. Bakılan karakter, bozulmanın olduğu karakterdir.

7.2 İkinci Durum: Karakter Desende Var

Bu kez hizalamada `E-E`, `L-L` uyuşur, `D` ile `N` uyuşmaz. Çakışan harfler üst üste gelecek şekilde `i` ayarlanmalıdır.

Peki aynı harf desende birden fazla geçiyorsa? Hoca, sınıftaki soruya cevap olarak bunu Horspool’un önerisi diye aktardı: Hepsiyle tek tek uğraşılmaz, yalnız harfin **en sağda** geçtiği yere (rightmost occurrence) bakılır. `NEEDLE`’daki üç `E` için yalnız son `E` dikkate alınır. Böyle yapınca aranan kelime arada kaçırılmaz. Atlama miktarı şudur:

$$\text{skip} = j - (\text{karakterin desende en sağda geçtiği konum})$$

Örnekte $i = 6$, $j = 3$ ve `N`'nin konumu 0'dır:

$$\text{skip} = 3 - 0 = 3, \qquad i = 6 + 3 = 9$$

Karakterin desenin ortasında olması da mümkündür. Bozan karakter `L` olsaydı, desen `L`'ler çakışacak şekilde yerleştirilirdi.

7.3 Atlama Negatif Çıkarsa

Hoca yavaş çekimle bir örnek daha gösterdi. $i = 16$, $j = 3$ olsun. `E` ve `L` uyuşmuş, metindeki `E` ile desendeki `D` uyuşmamış olsun. `E`'nin en sağdaki konumu 5'tir:

$$\text{skip} = 3 - 5 = -2, \qquad i = 16 - 2 = 14$$

Yani `i` geri gidiyor. Bunu istemeyiz. Çünkü geri gitmenin sonu yoktur; nereye kadar gideceğimiz belli olmaz. Sorun, bozan karakterin desendeki en sağ konumunun `j`'nin sağında kalmasıdır ($j = 3$ iken konum 4 veya 5 ise). Çözüm basittir:

> **Hocanın vurgusu:** Hesap sonucunda atlama negatif çıkarsa `i`'yi bir artırarak yola devam ederiz: skip $= 1$.

Örnekte yeni `i` 14 değil 17 olur.

> **Ek bilgi (derste anlatılmadı):** En sağdaki geçişi seçmek, seçenekler içinde en küçük kaydırmayı verir, bu yüzden eşleşme atlanmaz. Soldaki geçişlere göre kaydırmak daha çok atlatır. Ayrıca `skip` hiç 0 çıkmaz: $j - \text{table}[c] = 0$ olması `p[j] == c` demektir, oysa `c` bu konumda uyuşmamıştır. Bu nedenle kodda yalnız `skip < 0` denetimi yeterlidir ve döngü sonsuza girmez.

8. En Sağdaki Geçiş Tablosu

8.1 Sürekli Yapılan Gereksiz İşlem

Hoca, az önceki optimizasyon sözlerine bağlanarak bir soru sordu: burada sürekli, gereksiz yere yapılan şey nedir?

Soru (öğrenci önerisi): Son karakter uyuşunca sola gitmek yerine en baştaki karakter uyuşuyor mu diye baksak olur mu?

Cevap: Hocanın aradığı bu değildi. Metin çok uzunsa fena olmayabilir ama "baştaki uyuyorsa ne yapacaksın, ikinciye mi bakacaksın?" diye gider, sonu yoktur.

Bir öğrenci sonunda doğru cevabı verdi: Sürekli aramak yerine bir tür harita çıkarmak. Her uyuşmazlıkta desenin içinde o karakter var mı, varsa kaçıncı karakter diye bakıyoruz. Oysa desen program boyunca değişmez. `N`, `NEEDLE`'ın daima 0. karakteridir, burada sürpriz yoktur. Dolayısıyla bu bilgi başta bir kez bir tabloya yazılır, algoritma çalıştığı sürece tablodan okunur.

> **Ders sonrası not:** Hoca, bu tabloyu Boyer ve Moore'un değil Horspool'un fark ettiğini söyledi. Büyük olasılıkla özgün Boyer-Moore algoritmasında da en sağdaki geçiş için böyle bir tablo vardır. Büyük olasılıkla Horspool'un değişikliği, atlamayı uyuşmayan karaktere göre değil, her zaman desenin son karakterinin hizasındaki metin karakterine göre belirlemektir. O kuralda tablo ilk $m - 1$ karaktere göre kurulur, bu yüzden atlama hiç negatif çıkmaz. Derste anlatılan ve aşağıda kodu verilen `skip = j - table[c]` kuralı ise büyük olasılıkla Boyer-Moore'un kötü karakter (bad character) kuralıdır.

8.2 Tablonun Boyu

Hoca tahtada basitçe `A`'dan başlayan bir büyük harf tablosu kullandı. Gerçekte bütün ASCII veya Unicode karakterler için böyle bir tablo yapılır. Derste yalnız büyük harf geçtiği için 26 göz yeterlidir; 26, Latin alfabesinin büyük harf sayısı olarak söylendi. Tablonun boyu koddaki sembol sayısıdır.

Soru: Tablo yalnız desendeki harfleri mi, yoksa bütün sembol kümesini mi içeriyor?

Cevap: Bütün sembol kümesini. Sorunun haklı yanı da var: Kod çok uzunsa, örneğin 10 bin farklı sembol varsa, yalnız geçen karakterler tutulabilir ve hash tablosu gibi bir yapı kullanılabilir. ASCII gibi küçük kodlarda ise bütün tabloyu ayırmak kolaydır.

> **Ders sonrası not:** ASCII tablosunun boyu derste 128 mi 255 mi diye konuşuldu. Bir öğrencinin bakıp söylediği gibi, hocanın da ilk hatırladığı gibi ASCII 128 karakterdir.

8.3 Göz Numarasını Bulmak

Bir karakterin gözünü bulmak için `if` kullanılmaz. Karakterden bir kayma (offset) çıkarılır. Büyük harfler için `offset = 'A'`'dır: `'B' - 'A' = 1` olur, yani `B` 1 numaralı gözdedir. `'N' - 'A' = 13` olur. Çalışılan koda göre offset de değişir.

8.4 Tabloyu Doldurmak

Önce bütün gözlere -1 yazılır. Bu, karakterin desende geçmediğini gösterir. Sonra desen soldan sağa gezilir ve her karakterin gözüne konumu yazılır. `NEEDLE` için:

`i`	Karakter	Yapılan	Göz (`karakter - 'A'`)
0	`N`	`N`'nin gözüne 0 yazılır	13
1	`E`	`E`'nin gözüne 1 yazılır	4
2	`E`	`E`'nin gözündeki 1'in üzerine 2 yazılır	4
3	`D`	`D`'nin gözüne 3 yazılır	3
4	`L`	`L`'nin gözüne 4 yazılır	11
5	`E`	`E`'nin gözündeki 2'nin üzerine 5 yazılır	4

Soldan sağa gidip üzerine yazmak, en sağdaki geçişi kendiliğinden sağlar. Sonuç tablosu şudur:

Karakter	`N`	`E`	`D`	`L`	Diğerleri
En sağdaki konum	0	5	3	4	-1

Hoca, akış çizmeye gerek olmadığını söyleyip bunun kodunu yazdı. Tablo bir kez oluşturulur. `size` tablonun, yani kodun uzunluğudur:

```
void shiftTable(char pattern[], int table[], int offset, int size)
{
    int i, M = strlen(pattern);
    for (i = 0; i < size; i++)
        table[i] = -1;
    for (i = 0; i < M; i++)
        table[pattern[i] - offset] = i;
}
```

9. Boyer-Moore Kodu

Öğrencilerin tercihiyle hoca algoritmayı akış diyagramı yerine kod olarak yazdı. Fonksiyon desen `p` ve metin `t`'yi alır.

9.1 Kodun Parçaları

For döngüsü: `i` en son `N - M`'ye kadar gider. Metinde 3 karakter kalmışken 8 karakterlik bir dizgi aranmaz. `i`'nin artımı, `skip` ne çıkıyorsa odur. Hoca, başlangıç, artım ve bitişin aynı satırda görülsün diye burada `for` kullandığını söyledi.

> **Hocanın vurgusu:** Burada `for` kullanabiliyorum, çünkü fonksiyondan `return` ile dönüyorum. `return` ile dönmüyorsanız `for` içinden `break` ile çıkmayın. Anlaşılırlığı bozar. Bunun yerine `found` gibi bir bayrak kullanın.

j'nin başlangıcı: Desenin son harfi, yani `M-1`'inci göz.

Çıkışta kontrol: Döngüden niye çıktık? Bulduk ya da bulamadık:

- `j >= 0` ise eşitlik bozulmuştur, desen bu hizada yoktur.
- `j == -1` ise bütün karakterlere bakılmış, hepsi eşit çıkmıştır. Desen bulunmuştur ve başladığımız yerin adresi `i` döndürülür.

Atlama: Eşitlik `t[i+j]` karakterinde bozulmuştur. Hoca burada tablodaki -1'in güzelliğini gösterdi. Tek formül iki durumu da karşılar:

$$\text{skip} = j - \text{table}[c]$$

Karakter desende yoksa $j-(-1) = j+1$ olur; bu tam istediğimiz şeydir. Varsa j eksi karakterin en sağ konumu olur. Geriye gitmesin diye `skip < 0` ise `skip = 1` yapılır.

Bulunamama: For bitmiş ve hiçbir hizada desen bulunmamışsa adres döndüremeyeceğimiz için `-1` (not found) döndürülür.

9.2 Tam Program

Tahtadaki fonksiyonda tablo indisi `table[t[i+j]]` olarak yazılıdır. Derste büyük harf tablosu kullanıldığı için aşağıda `offset` çıkarılmış hâli verilmiştir. Tablonun global tanımı ve `main`, derlenebilmesi için eklenmiştir:

```c
#include <stdio.h>
#include <string.h>

int table[26];

void shiftTable(char pattern[], int table[], int offset, int size)
{
    int i, M = strlen(pattern);
    for (i = 0; i < size; i++)
        table[i] = -1;
    for (i = 0; i < M; i++)
        table[pattern[i] - offset] = i;
}

int BoyerMoore(char p[], char t[])
{
    int i, j, skip = 0, N = strlen(t), M = strlen(p);
    for (i = 0; i <= N - M; i += skip)
    {
        j = M - 1;
        while ((j >= 0) && (p[j] == t[i + j]))
            j--;
        if (j >= 0)                    /* pattern bulunamadi */
        {
            skip = j - table[t[i + j] - 'A'];
            if (skip < 0)
                skip = 1;
        }
```

```
        else
            return i;               /* p'nin basladigi adres */
    }
    return -1;                      /* not found */
}

int main(void)
{
    char t[] = "FINDINAHAYSTACKNEEDLEINA", p[] = "NEEDLE";
    shiftTable(p, table, 'A', 26);
    printf("N=%d E=%d D=%d L=%d S=%d\n", table['N' - 'A'], table['E' - 'A'],
           table['D' - 'A'], table['L' - 'A'], table['S' - 'A']);
    printf("%d\n", BoyerMoore(p, t));
    return 0;
}
```

Program `gcc -Wall` ile uyarısız derlendi. Çıktısı `N=0 E=5 D=3 L=4 S=-1` ve `15` oldu. Bu sonuçlar tahtadaki tablo ve izlemeyle aynıdır.

> **Ders sonrası not:** Bu kodun `for` satırında `N - M` her turda hesaplanır. Hocanın Bölüm 4.2'deki önerisi burada da geçerlidir: değer döngüden önce bir değişkene atanabilir.
>
> **Ek bilgi (derste anlatılmadı):** `t[i+j] - 'A'` indisi yalnız metin büyük harflerden oluşuyorsa geçerlidir. Boşluk veya küçük harf gelirse indis tablonun dışına çıkar. Tablo karakterle doğrudan indislenecekse 128 ya da 256 gözlü olmalıdır. `char` işaretli olabildiğinden indis `(unsigned char)t[i+j]` diye alınmalıdır.

10. Örnek Üzerinde Adım Adım İzleme

Hoca algoritmayı aynı örnekte izledi. Metin `FINDINAHAYSTACKNEEDLEINA`, desen `NEEDLE`, $m = 6$, tablo Bölüm 8.4'tekidir. İlk karşılaştırma `E` iledir, yani $j = M - 1 = 5$.

`i`	`j`	Karşılaştırma	Bozan karakter	`skip`	Yeni `i`
0	5	`t[5]='N'` ile `E`	`N` (tabloda 0)	$5 - 0 = 5$	5
5	5	`t[10]='S'` ile `E`	`S` (tabloda -1)	$5 - (-1) = 6$	11
11	4	`E`-`E` uyuştu, `t[15]='N'` ile `L`	`N` (tabloda 0)	$4 - 0 = 4$	15
15		`E, L, D, E, E, N` hepsi uyuştu			`return 15`

Hizalamalar yan yana şöyle görünür:

```
indis : 000000000011111111112222
        012345678901234567890123
metin : FINDINAHAYSTACKNEEDLEINA
i=0   : NEEDLE
i=5   :      NEEDLE
i=11  :            NEEDLE
i=15  :                NEEDLE
```

Toplamda 4 hizalamada $1 + 1 + 2 + 6 = 10$ karakter karşılaştırması yapılır (ara adım eklendi). Aynı aramada kaba kuvvet 23 karşılaştırma yapar. Her iki sayı da sayaçlı çalıştırmayla doğrulandı. Hizalama sayısı $n/m = 24/6 = 4$ ile bu örnekte tam olarak uyuşur.

> **Ders sonrası not:** Hoca ikinci hizalamada "i 5, j 10" dedi; doğrusu $i = 5$, $j = 5$'tir. 10, karşılaştırılan metin konumu olan $i + j$'dir.

11. Boyer-Moore Karmaşıklığı

11.1 Normal Metin

İyi şartlarda yaklaşık n/m karşılaştırma yapılır. Hoca bunun en iyi durum olmaktan öte, ortalama genel durum için söylendiğini belirtti. Normal bir yazıda, tekrar çok değilse, uyuşmazlık genelde bir iki karakter sonra çıkar. Kaba kuvvet iyi durumda bile n karşılaştırma yaparken bu yöntem normal bir metinde yaklaşık n/m karşılaştırma yapar.

11.2 En Kötü Durum

Hoca az önceki gibi en kötü durumu da gösterdi. Metnin tamamı `B`, desen `ABBBB` (bir `A`, dört `B`) olsun. Sondan başa gidilir: `B`-`B`, `B`-`B`, `B`-`B`, `B`-`B` uyuşur. En sonda `A`-`B` uyuşmaz. Bu bir hizada m karşılaştırma demektir. Bozan karakter `B`'dir. Atlama negatif çıkacağından yalnız 1 kaydırılabilir. Sonra aynı şey tekrarlanır. Sonuç olarak en kötü durum burada da $O(n \cdot m)$'dir.

> **Ders sonrası not:** Hoca bir kaydırmanın nedeni için "karakter desende bulunmadığı için" dedi; doğrusu `B` desende vardır. Bozan `B` $j = 0$'dadır, en sağdaki `B` konum 4'tedir. skip $= 0 - 4 = -4$ negatif çıkar ve 1'e çekilir. Bu örnekte toplam karşılaştırma tam olarak $(n - m + 1) \cdot m$'dir; farklı n değerleriyle sayaçlı çalıştırma bunu doğruladı.

Hoca bu durumu olağan bir yazıda beklemediğimizi vurguladı. Desenin $m-1$ karakterinin her seferinde uymasını, m'inci kontrolde hata oluşmasını gerektirir. Böyle veriler daha çok DNA dizilerinde çıkabilir. Bu yüzden normal metinde bu yöntem kaba kuvvetten avantajlıdır.

Daha da avantajlı arama yaklaşımları vardır. Hoca bunların 3. sınıftaki Algoritma Analizi dersinde işleneceğini söyledi. O dersin üç hedefi vardır:

- Özyinelemeli olan ve olmayan daha karmaşık algoritmaların analizi ile etkili algoritma yazmadaki eksikleri tamamlamak.
- Farklı algoritma türleri görmek.
- Bu dersteki temel konuların üzerine, gerçek dünyada sık kullanılan yöntemleri eklemek. Hocanın örneği, graf üzerinde en kısa yol (shortest path) algoritmalarıdır.

12. Trie ve Otomatik Tamamlama

Hoca konuyu bitirirken sevdiği yapılardan birini "zevkli bir şey" diye gösterdi: WhatsApp gibi uygulamalarda yazarken kelime ya da cümlenin tamamlanması. Bunun arkasında trie (önek ağacı) denen bir ağaç yapısı vardır.

Önce bir sözlük (dictionary) vardır ve bu sözlük ağaca yerleştirilir. Kullanıcı yazmaya başladıkça ağaçta ilerlenir ve o noktadan devam eden kelimeler önerilir. Örnek sözlük şudur: `by`, `sea`, `sells`, `she`, `shells`, `shore`, `the`.

- Kullanıcı `s`, `h` yazınca `s` → `h` düğümüne gelinir. Altındaki kelimeler (`she`, `shells`, `shore`) önerilir.
- `o` da yazılırsa yalnız `shore` kalır, doğrudan o önerilir.
- `b` yazılınca sözlükte bu harfle başlayan tek kelime `by` olduğu için hemen `by` önerilir.

Hoca bu yapının dizgi aramalarında kullanıldığını söyleyerek konuyu kapattı.

Dersin Toparlaması

Bu derste bir metinde bir desenin tamamını arama problemi ele alındı. Kaba kuvvet, her konumdan soldan sağa karşılaştırır ve uyuşmazlıkta bir sağa kayar. Normal metinde yaklaşık n karşılaştırma yaparken, tekrarlı verilerde en kötü durumu $O(n \cdot m)$ olur. Boyer-Moore yaklaşımı ise sağdan sola karşılaştırır ve bozan karakterin desende nerede geçtiği bilgisini kullanarak, eşleşmeyi kaçırmadan daha çok atlar. Atlama tek formülle hesaplanır: skip $= j - \text{table}[c]$. Negatif çıkarsa 1 alınır. En sağdaki geçişler başta bir kez hesaplanıp tabloya yazılır. Böylece

normal metinde karşılaştırma sayısı yaklaşık n/m'ye düşer, en kötü durum ise yine $O(n \cdot m)$ kalır.

Dersin bir başka mesajı, tasarım alışkanlıklarıydı: önce elle örnek denemek, sonra değişmeyen hesapları döngü dışına, değişmeyen bilgileri tabloya almak. Son olarak trie yapısıyla dizgi aramanın otomatik tamamlamadaki kullanımı gösterildi. Daha ileri dizgi arama yöntemleri ve graf üzerinde en kısa yol algoritmaları, 3. sınıftaki Algoritma Analizi dersinde ele alınacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.